

Linux 分布式监控系统 — 校招项目文档

适用对象：校招应届生 / 实习求职

项目类型：C++ 后端 / 系统监控

技术栈：Modern C++17、gRPC、Protocol Buffers、Linux /proc 文件系统、Python Web Dashboard

一、项目介绍

1.1 项目背景

Linux 服务器在日常运行中，CPU、内存、网络、中断等系统指标直接反映机器的健康状态。运维人员和开发者需要一套**低开销、高精度、可扩展**的监控系统来实时掌握这些指标。

本项目实现了一套**分布式 Linux 性能监控系统**，核心用 Modern C++17 编写，通过读取 Linux /proc 文件系统采集系统指标，使用 gRPC + Protocol Buffers 实现客户端-服务端数据传输，并提供 Python Web Dashboard 做可视化展示。

1.2 项目定位

维度	说明
项目类型	系统监控 + 分布式通信
代码规模	~2000 行核心 C++，多个独立模块
适用场景	单机监控、多机集群监控、开发调试
技术深度	OS 内核接口、RPC 框架、序列化协议、设计模式

1.3 核心功能

- CPU 监控：**从 /proc/loadavg 读负载，从 /proc/stat 读 8 种 CPU 状态并做差分计算
- 内存监控：**从 /proc/meminfo 解析 19 个内存指标，含使用率计算和单位换算
- 网络监控：**从 /proc/net/dev 读各网卡收发字节/包数，做速率差分

- **软中断监控**: 从 `/proc/softirqs` 读 10 类软中断在各 CPU 核上的速率
- **RPC 通信**: gRPC 同步调用, 客户端采数据 → 服务端缓存 → Web 端拉取
- **Web Dashboard**: Python 实现, 多页面实时展示各项指标

二、项目结构

```
linux_monitor/
├── linux_monitor/           # 监控客户端（数据采集）
│   ├── include/
│   │   ├── monitor/       # 监控器接口 + 各实现类头文件
│   │   └── utils/         # 文件读取工具、时间工具
│   └── src/
│       ├── main.cpp        # 客户端入口, 定时调度
│       ├── monitor/        # 各监控器 .cpp 实现
│       └── utils/          # 工具类实现
├── rpc_manager/            # RPC 通信层
│   ├── client/             # gRPC 客户端封装
│   ├── server/             # gRPC 服务端实现
│   └── CMakeLists.txt
├── proto/                  # 数据协议定义 (.proto 文件)
│   ├── monitor_info.proto  # 顶层聚合消息 + RPC 服务定义
│   ├── cpu_load.proto
│   ├── cpu_stat.proto
│   ├── cpu_softirq.proto
│   ├── mem_info.proto
│   └── net_info.proto
├── web_dashboard/          # Web 可视化 (Python)
│   ├── main.py
│   └── requirements.txt
├── docker/                 # Docker 容器化部署
│   └── scripts/
├── build/                  # 构建产物
└── CMakeLists.txt          # 顶层构建配置
```

模块依赖关系:



三、数据结构

3.1 Protocol Buffers 消息定义（核心数据协议）

```
// 顶层消息：一次采集包含所有监控数据
message MonitorInfo {
    CpuLoad cpu_load = 1;           // CPU 负载
    repeated CpuStat cpu_stat = 2;  // 每核 CPU 状态
    repeated CpuSoftirq cpu_softirq = 3; // 每核软中断
    MemInfo mem_info = 4;           // 内存指标
    repeated NetInfo net_info = 5;  // 每网卡流量
}

// CPU 状态消息
message CpuStat {
    string cpu_name = 1;           // "cpu" 或 "cpu0"/"cpu1"...
    float cpu_percent = 2;         // 总体使用率 %
    float usr_percent = 3;         // 用户态 %
    float system_percent = 4;      // 系统态 %
    float nice_percent = 5;        // 低优先级 %
    float idle_percent = 6;        // 空闲 %
    float io_wait_percent = 7;     // I/O 等待 %
    float irq_percent = 8;         // 硬中断 %
    float soft_irq_percent = 9;    // 软中断 %
}

// RPC 服务定义
service MonitorService {
    rpc SetMonitorInfo(MonitorInfo) returns (google.protobuf.Empty);
    rpc GetMonitorInfo(google.protobuf.Empty) returns (MonitorInfo);
}
```

设计考量：

- 用 `repeated` 字段天然支持多核 CPU 和多网卡，无需硬编码数量
- 顶层 `MonitorInfo` 聚合所有子消息，一次 RPC 调用传完整快照，减少网络往返
- 字段编号从 1 开始连续分配，保证二进制兼容性

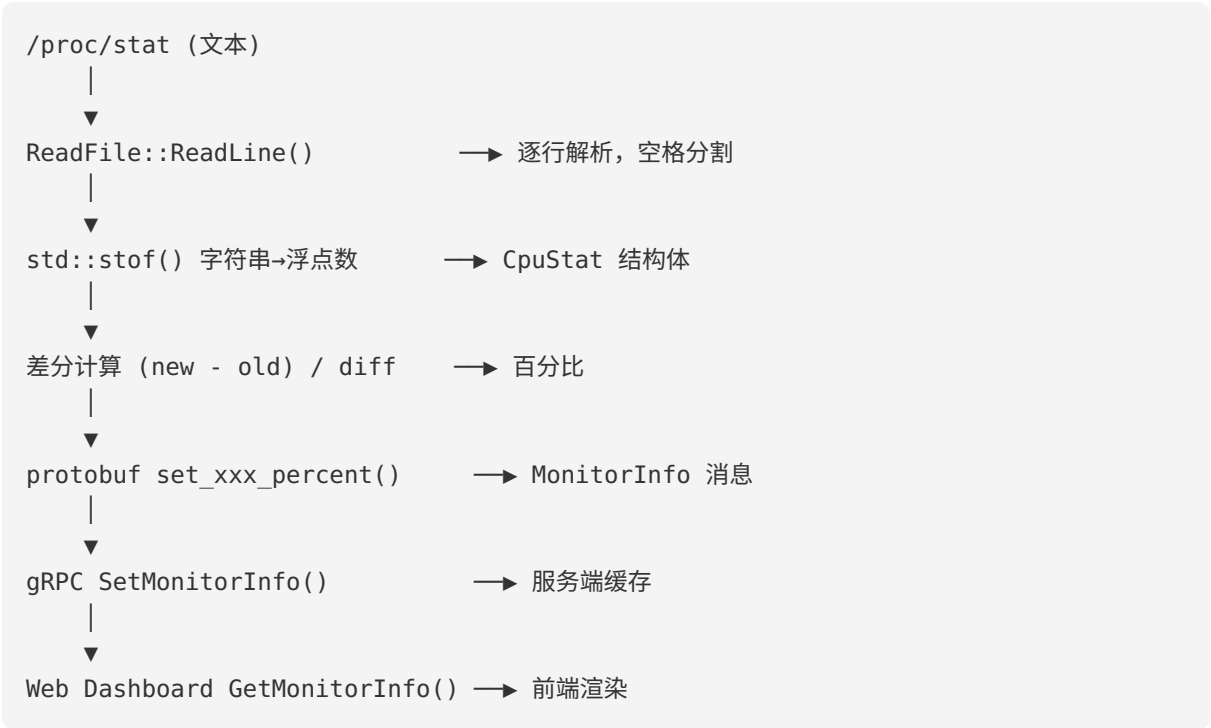
3.2 内存中的核心结构

```
// CPU 状态 — 对应 /proc/stat 每行 10 个字段
struct CpuStat {
    std::string cpu_name;
    float user, nice, system, idle;
    float io_wait, irq, soft_irq;
    float steal, guest, guest_nice;
};

// 差分计算引擎 — 存储上一次采样值
```

```
std::unordered_map<std::string, CpuStat> cpu_stat_map_;  
// key: "cpu" / "cpu0" / "cpu1" ...  
// value: 上次采样的原始 jiffies 值
```

3.3 数据流



四、技术细节

4.1 Linux /proc 文件系统读取

所有监控数据均来自 Linux 内核暴露的 `/proc` 伪文件系统：

文件	内容	更新机制
<code>/proc/loadavg</code>	1/5/15 分钟平均负载	内核实时计算
<code>/proc/stat</code>	各 CPU 核的时间片累计值 (jiffies)	每次时钟中断累加
<code>/proc/meminfo</code>	内存详细统计 (KB)	内核实时更新
<code>/proc/net/dev</code>	各网卡收发字节/包累计值	每次收发更新
<code>/proc/softirqs</code>	各核各类软中断累计次数	每次中断累加

关键点： `/proc/stat`、`/proc/net/dev`、`/proc/softirqs` 提供的都是**开机以来的累计值**，不能直接使用，需要做差分计算。

4.2 差分计算算法

以 CPU 使用率为例，核心流程：

```
第 1 次采样 (t=0): cpu_total = 100000 jiffies
                    cpu_busy  = 30000  jiffies
                    → 存到 map, 不输出

第 2 次采样 (t=3): cpu_total = 100300 jiffies    (增加了 300)
                    cpu_busy  = 30120  jiffies    (增加了 120)
                    → diff_total = 100300 - 100000 = 300
                    → diff_busy  = 30120  - 30000  = 120
                    → cpu_percent = 120 / 300 * 100 = 40.0%
                    → 更新 map, 输出结果
```

为什么用两次采样而不是单次？ 因为 `/proc/stat` 里的值是累计值，只有差值才能反映"最近一段时间内"的 CPU 占用情况。

4.3 RAII 文件读取封装

```
class ReadFile {
public:
    explicit ReadFile(const std::string& name) : ifs_(name) {}
    ~ReadFile() { ifs_.close(); }           // 自动关闭
    bool ReadLine(std::vector<std::string>* args); // 读一行并分割
private:
    std::ifstream ifs_;
};
```

构造函数打开文件，析构函数自动关闭。即便 `ReadLine` 中途抛异常，栈展开也会调析构函数释放文件句柄，不会泄漏。

4.4 策略模式 — 监控器框架

```
// 抽象接口
class MonitorInter {
public:
    virtual void UpdateOnce(monitor::proto::MonitorInfo* monitor_info) = 0;
    virtual void Stop() = 0;
    virtual ~MonitorInter() {}
};

// 具体实现
```

```
class CpuStatMonitor : public MonitorInter { ... };
class MemMonitor : public MonitorInter { ... };
class NetMonitor : public MonitorInter { ... };
class CpuSoftirqMonitor : public MonitorInter { ... };
```

客户端主循环中，所有监控器存在一个 `vector<unique_ptr<MonitorInter>>` 里，依次调用 `UpdateOnce()`。新增监控指标只需要再写一个子类，**对扩展开放，对修改封闭（开闭原则）**。

4.5 时间基准选择

整个项目的计时间隔全部使用 `std::chrono::steady_clock`，而不是 `system_clock`：

时钟类型	特点	适合场景
<code>system_clock</code>	系统时间，可被 NTP/手动调整	显示时间戳
<code>steady_clock</code>	单调递增，永不回退	这里： 计算时间间隔

如果用了 `system_clock`，当 NTP 校时导致时间跳变，差分计算的分母（时间间隔）就会出现负值或极大值，输出错误数据。

五、技术难点与解决方案

难点 1：累计值的实时速率计算

问题： `/proc/stat`、`/proc/net/dev`、`/proc/softirqs` 给出的都是系统启动以来的累计值。如果直接除以开机时间，得到的是"历史平均"，而不是"当前实时"。

解决方案：

- 维护 `unordered_map<string, CpuStat>` 存储上一次采样的原始累计值
- 每次采样计算 `(new - old) / time_diff`，得到的是两次采样间隔内的**瞬时速率**
- 首次采样只存不输出，保证有旧数据可用

难点 2：多核 CPU 的动态解析

问题：不同机器的 CPU 核心数不同（4 核、8 核、16 核），`/proc/stat` 的行数不固定。

解决方案：

- 用 `repeated` Protobuf 字段，按实际核心数动态填充

- 用 `unordered_map` 按 `cpu_name` 做 key，核心数变化（热插拔）也能自然处理
- 不硬编码核心数，代码天然跨机器兼容

难点 3: `guest` 和 `guest_nice` 字段的重复计数

问题：Linux 内核文档明确指出，`/proc/stat` 中的 `guest` 时间已经包含在 `user` 时间里。如果直接全部求和，会重复计算。

解决方案：计算总时间片时**排除** `guest` 和 `guest_nice`：

```
float new_cpu_total_time = cpu_stat.user + cpu_stat.system +
    cpu_stat.idle + cpu_stat.nice +
    cpu_stat.io_wait + cpu_stat.irq +
    cpu_stat.soft_irq + cpu_stat.steal;
// 注意：没有加 guest 和 guest_nice
```

难点 4: Protobuf 字段的实际使用率追踪

问题：项目从最初只监控 CPU 逐步扩展到 CPU+内存+网络+软中断，protobuf 消息和 RPC 接口也在不断增加字段。Protobuf 的向后兼容机制（字段编号永不变、新增字段用新编号）确保了：

- 老客户端发的消息，新服务端能解析（未知字段被忽略）
- 新客户端发的消息，老服务端能解析（缺少的字段用默认值）

这其实是 Protobuf 的天然特性，但在项目中实际体会到了它的价值。

难点 5: 避免除零错误

问题：当两次采样的总时间差为 0（极短间隔或系统刚启动），直接做除法会得到 `inf` 或 `nan`。

解决方案：所有百分比计算都在 `if (total_time_diff > 0)` 条件下进行，杜绝除零隐患。

难点 6: 网络监控中的接口发现

问题：`/proc/net/dev` 前两行是表头，且机器可能有 `lo`、`eth0`、`docker0` 等多个接口。

解决方案：跳过前两行表头，后续每行按接口名解析到独立的 `NetInfo` 消息中。每个接口独立做差分，互不干扰。

六、项目亮点

6.1 架构设计能力

- **三层解耦**：数据采集层 → RPC 通信层 → Web 展示层，每层可以独立替换
- **策略模式**实现可插拔监控器，新增监控指标只需新增一个子类
- **接口隔离**：MonitorInter 只定义 UpdateOnce() 和 Stop() 两个方法，干净利落

6.2 系统编程功底

- 深入理解 Linux /proc 文件系统，知道每个文件的数据含义和更新机制
- 正确处理累计值 → 速率的差分转换，理解为什么不能直接读
- 理解 guest/guest_nice 的重复计数陷阱
- 选用 steady_clock 而非 system_clock 来计算时间间隔

6.3 现代 C++ 实践

- RAII 管理文件句柄，杜绝资源泄漏
- 智能指针 unique_ptr 管理对象生命周期
- unordered_map 做 O(1) 查找的历史数据存储
- 使用 std::chrono 库做时间计算

6.4 工程化能力

- CMake 模块化构建，每个子目录独立 CMakeLists.txt
- Protocol Buffers 定义清晰的数据协议，强类型，跨语言
- gRPC 实现客户端-服务端通信，支持分布式部署
- Docker 容器化，一键构建运行
- .proto 文件是跨语言的单一数据源（C++ 服务端 + Python Web 都能用）

6.5 完整可运行

- 不是 demo，是真正能跑起来的系统：./bin/server + ./monitor + web dashboard 三端配合
 - 单次采集数据量仅 ~1.1KB，CPU 占用 <2%，内存 ~15MB，不影响被监控系统
 - 3 秒采集间隔，10ms 内完成一次采集
-

七、面试常问八股（结合本项目）

7.1 C++ 相关

Q: virtual 析构函数为什么要加？

A: 本项目中 `MonitorInter` 是抽象基类，外部通过 `vector<unique_ptr<MonitorInter>>` 持有子类对象。如果析构函数不是 `virtual`，`delete` 基类指针时只会调基类析构，子类析构被跳过，造成资源泄漏。加上 `virtual ~MonitorInter() {}` 后，析构会从子类一路调到基类。

Q: RAII 是什么？项目里怎么用的？

A: Resource Acquisition Is Initialization，资源获取即初始化。本项目 `ReadFile` 类在构造函数中打开文件 (`ifs_(name)`)，在析构函数中关闭文件 (`ifs_.close()`)。对象离开作用域时自动释放文件句柄，即使发生异常也不会泄漏。

Q: unique_ptr 和 shared_ptr 的区别？你为什么用 unique_ptr？

A: `unique_ptr` 独占所有权，不可拷贝，只可移动。`shared_ptr` 共享所有权，内部有引用计数。本项目中监控器对象的生命周期由客户端主循环独占管理，没有共享需求，用 `unique_ptr` 就够，零额外开销。

Q: unordered_map 的底层实现？

A: 哈希表。平均 $O(1)$ 查找。本项目用它做 `cpu_stat_map_`，key 是 CPU 名称（如 "cpu0"），查找上一次采样数据只需一次哈希运算。与 `map`（红黑树， $O(\log n)$ ）相比，这里不需要有序遍历，选 `unordered_map` 更快。

7.2 操作系统相关

Q: /proc 文件系统是什么？

A: Linux 内核提供的虚拟文件系统，本质是内核数据结构的访问接口。读取 `/proc/stat` 时，内核实时计算并返回数据，文件并不真实存储在磁盘上。它提供了用户态程序和内核态数据之间的桥梁。

Q: 用户态和内核态的区别？你读 /proc/stat 的过程发生了什么？

A: 用户态是受限的 CPU 运行级别，不能直接访问硬件和内核数据结构。调用 `std::ifstream` 读 `/proc/stat` 时，底层触发 `open()` $\rightarrow$ `read()` 系统调用，CPU 从用户态切换到内核态，内核在 `/proc` 子系统中构造数据返回，再切回用户态。

Q: 硬中断和软中断的区别？

A: 硬中断由硬件（网卡、磁盘）产生，打断 CPU 当前执行流，要求立即处理。软中断是内核在硬中断处理中延后执行的部分，用于减少硬中断的关中断时间。本项目监控 `/proc/softirqs` 中的 `NET_RX/NET_TX` 软中断量，高值意味着网络数据包处理繁忙。

Q: CPU 使用率的"使用"到底指什么?

A: CPU 在任意时刻要么在执行指令 (user/system/nice/irq/softirq/steal) , 要么在空闲 (idle) 或等 I/O (iowait) 。CPU 使用率 = 非空闲时间 / 总时间。本项目从 `/proc/stat` 的时间片差值计算这个比例。

7.3 网络/协议相关

Q: Protocol Buffers 比 JSON 好在哪里?

A: 1) 二进制序列化, 体积小 (本项目单次采集 ~1.1KB) ; 2) 反序列化快, 无需解析文本; 3) 强类型 + schema 定义, 编译器生成类型安全的访问代码; 4) 向后兼容, 新增字段用新编号不会破坏老代码。

Q: gRPC 和 REST 的区别?

A: gRPC 基于 HTTP/2 + Protobuf, 支持双向流、连接复用、头部压缩。REST 基于 HTTP/1.1 + JSON, 文本可读但带宽大。对于监控系统这种高频小数据量场景, gRPC 更合适。

Q: 为什么定义 `SetMonitorInfo` 和 `GetMonitorInfo` 两个 RPC, 而不是一个?

A: 职责分离。客户端采数据后调 `Set` 推送给服务端; Web Dashboard 调 `Get` 从服务端拉取。两者调用频率、调用方都不同, 拆成两个接口更清晰。

八、后续可扩展方向

方向	技术方案	难度
磁盘 I/O 监控	<code>/proc/diskstats</code> 解析	低
进程级监控	<code>/proc/<pid>/stat</code> + <code>/proc/<pid>/status</code>	中
历史数据存储	InfluxDB / Prometheus 时序数据库	中
告警引擎	阈值规则 + 钉钉/邮件通知	中
eBPF 深度监控	eBPF 内核探针, 监控系统调用、网络包	高
分布式追踪	OpenTelemetry 集成	高

附录：校招写项目文档的建议

1. 项目介绍不要只写"我做了什么", 要写"解决了什么问题"——给面试官一个场景感

2. **技术细节是区分度最高的部分**——不要只写用了什么技术，要写**为什么这样用**以及**遇到了什么问题**
3. **难点和解决方案一节是面试官最爱问的**——要诚实，不是每个难点都必须"解决"了，也可以写"目前的做法和它的局限性"
4. **数据结构一节展示你的抽象能力**——你是如何把现实世界的概念映射到代码中的结构体和消息的
5. **八股结合项目回答**——面试官问 "virtual 析构函数"，你不光背定义，还能说"在我们项目的 MonitorInter 基类里就是这么用的"，效果翻倍
6. **量化一切可以量化的**——代码行数、采集间隔、CPU 占用、内存占用、数据体积，数字让项目显得真实且专业